ORDER FLOW PLATFORM

Complete Technical Audit

Volume Profile Terminal + Context Engine (NQ)

Read-only audit · git b3217da · 261 tests passing
Dataset: 62 parquet days (NQM6/NQU6, 2026-05-04 -> 2026-08-06)
No code modified.

Volume Profile Terminal + Context Engine (NQ)

Read-only audit - based on real code at git `b3217da` - dataset: 62 parquet days (NQM6/NQU6, 2026-05-04 -> 2026-08-06)

Scope: audit + documentation only. No code was modified.

> ***Golden rule of the product (verified respected across the codebase):*** *the tool is *context analysis*, never BUY/SELL, scores, probabilities, ML or a trade verdict. "This is what order flow is doing where you are. YOU decide if your price-action setup is worth executing."*

---

1. EXECUTIVE SUMMARY

The platform is a **desktop Volume-Profile + order-flow terminal for NQ** with a **Context Engine** that turns trades data into **11 descriptive components + one transparent `overall` summary**. Engineering-wise it is **mature and disciplined**: pure calculation engines in `core/` are cleanly separated from the Qt UI in `app/desktop/`, and the whole context layer is a **pure, deterministic, causal function** of a physically-truncated `Snapshot`.

What is excellent:

- **No-look-ahead by construction** -- the strongest part of the project. Confirmed in code, in tests (the "identical result whether or not the future exists" pattern), and on real data (replay path == day-slice path at the same T). No real look-ahead was found.
- **Correct aggressor mapping** (`B`=buy/+delta, `A`=sell/?delta), confirmed against the data (~50/50 split, N~0%).
- **Correct conceptual separations, tested with counter-examples**: absorption != "aggression without progress"; tape speed != direction; CVD divergence is swing-*structural*, not a naïve slope.
- **Determinism** and **high test quality** (many tests validate the *concept*, not just the implementation).

What is problematic (highest first):

#	Severity	Issue
1	MEDIUM	**90D composite silently drops the pre-rollover contract** -> "90D" ~ 45 real days (verified empirically).
2	MEDIUM	**Absorption/exhaustion implemented twice** (fixed-threshold legacy `detect_*` for markers vs adaptive `analyze_*` in the engine) -> two sources of truth that can disagree.
3	MEDIUM	`overall="SUPPORTIVE"` **is colored green for both bullish and bearish coherence** -> misleading (green=long in trading convention).
4	MEDIUM	`overall` **counts correlated evidence as independent** (bull-absorption + bot-exhaustion at a low = 2 "votes" from one phenomenon). Codified as intended by a test.
5	LOW/latent	**Fixed 22:00 UTC session boundary** is correct only in summer (EDT); winter (EST) needs 23:00 UTC. Not triggered by the May-Aug dataset.
6	LOW	`ts_recv` **used, not** `ts_event` -- capture jitter, negligible at >=1min bars.

Bugs: no correctness bug produces wrong numbers on the current data. One *latent* bug (DST boundary) and one *latent* fragility (cursor->index assumes contiguous bars -- verified there are no interior gaps on NQ) exist. The rest are design/UX/redundancy.

Most important next step: statistical empirical validation with a baseline -- not new code, not refactoring. Measure each component-state's follow-through across multiple horizons vs the *unconditional base rate*, over all 62 days, with MFE/MAE and lift-over-chance. Only that separates information from noise. Everything is currently **UNKNOWN as predictive value**.

---

2. ARCHITECTURE

2.1 Module map (project source, venv excluded)

```
data/loader.py            (268)  parquet/CSV reader, active-contract pick, 22:00 UTC session boundary
core/vp_engine.py         (366)  VolumeProfileEngine: POC/VA/HVN-LVN + VolumeNode + compute_nodes
core/delta_engine.py      (103)  DeltaEngine: buy/sell per level, CVD
core/context_engine.py   (1534)  * Snapshot + 11 components + _execution_summary (overall)
app/desktop/data_service.py(707) load_day->DayData; developing_levels; build_reference/composite_levels;
                                 legacy detect_absorption/detect_exhaustion (chart markers)
app/desktop/replay.py     (248)  tick-by-tick causal replay
app/desktop/charts.py     (616)  pyqtgraph render items (candles, profile, footprint, grid stats)
```

```
app/desktop/main.py      (2332)  Qt UI, Execution Context panel, hover/replay wiring
app/desktop/{theme,drawings}.py  cosmetic / drawing tools
scripts/validate_context.py(187) real-data validation harness (P7)
scripts/convert_to_parquet.py    CSV->parquet (uses loader.read_file)
scripts/{delta_analysis, multi_day_analysis, session_reanalysis,
         visualize_profile, load_databento_csv, live_simulator, verify_ohlc}  LEGACY standalone utilities
app/streamlit_app.py             LEGACY web UI (superseded by desktop)
tests/ (28 files, ~249 test fns)
```

2.2 Data-flow diagram

```
RAW (Databento GLBX.MDP3 trades CSV/DBN)
   |  scripts/convert_to_parquet.py -> read_file -> normalized [ts,price,size,side,symbol]
   v
PARQUET (data/parquet/*.parquet, all symbols kept)
   |  data/loader.py: _resolve_path -> _read_raw -> load_ticks/load_many
   |      - picks max-volume symbol (active contract)  - session grouping 22:00->22:00 UTC
   v
NORMALIZATION -> SessionTicks (one symbol, sorted by ts)
   |  app/desktop/data_service.py: load_day()
   |      - resample OHLC(interval)  - VolumeProfileEngine -> POC/VA/HVN/LVN
   |      - DeltaEngine -> buy/sell/CVD  - _build_footprint  - developing_levels (dev_poc/vah/val)
   |      - tps (trades/sec)  - legacy detect_absorption/exhaustion (markers)
   v
DayData  (full-day arrays + developing arrays + footprint)
   |  build_reference_levels (prev session + Asia/London/NY, available_from gated)
   |  build_composite_levels (15D/90D, ends yesterday)
   v
CAUSAL SNAPSHOT  (core/context_engine.snapshot_from_daydata, upto_index=k)
   |      - physically slices arrays to [0..k]  - uses dev_poc[k] (NOT day.poc)
   |      - footprint filtered to epochs <= T  - available_from <= now gates
   v
CONTEXT ENGINE  (ContextEngine.analyze -> 11 components + overall + reasons)   <- pure/deterministic
   |
   |--- REPLAY (app/desktop/replay.py) feeds ticks -> developing DayData -> same snapshot path
   v
UI  (main.py Execution Context panel, human-readable, hides NONE, disclaimer)
   v
USER  (discretionary read of context; decides execution with own price action)
```

Each stage explained:

- **RAW -> PARQUET**: one-time conversion; parquet is a full **UTC calendar day**, all symbols retained (including calendar spreads like `NQM6-NQU6`).
- **NORMALIZATION**: the loader is the *single source of truth* for reading; it does **no** analytics. Active contract = highest-volume symbol; spreads are filtered out because the outright dominates.
- **CALCULATIONS**: engines compute, never draw. `load_day` produces both **final** levels (full day, for display) and **developing** levels (causal, for the engine).
- **SNAPSHOT**: the anti-look-ahead boundary. Physically contains only <= T data.
- **CONTEXT ENGINE**: pure function; same snapshot -> same result.
- **REPLAY**: rebuilds the same DayData shape tick-by-tick; feeds the identical snapshot path.
- **UI**: renders context descriptively; markers are OFF by default (Clean Chart).

2.3 Architectural findings

Sev	Problem	Why / Impact	Evidence	Recommendation
MEDIUM	Absorption/exhaustion logic duplicated	`data_service.detect_absorption/exhaustion` (fixed thresholds, markers) vs `context_engine.analyze_absorption/exhaustion` (adaptive). Two definitions can disagree; a marker may contradict the panel.	`data_service.py:218-295` vs `context_engine.py:496-621`	Make the chart markers *consume* the engine result, or clearly label them as a different (fixed-threshold) view. Do not silently keep two.
LOW	Dead/legacy code in repo	`scripts/{load_databento_csv,multi_day_analysis,session_reanalysis,visualize_profile,delta_analysis}` + `streamlit_app.py` are superseded and imported by nothing in the app.	`grep`: no app import of scripts/streamlit	Move to `legacy/` or delete; keeps the surface honest.

Sev	Problem	Why / Impact	Evidence	Recommendation
LOW	Dependency direction is clean, one soft spot	`core/` depends only on numpy + `core.vp_engine` (good). `snapshot_from_daydata` accepts a DayData by **duck typing** so core stays UI-independent (good). `replay.py` imports the **legacy** detectors from `data_service`.	`context_engine.py:19-22`, `replay.py:15-16`	Acceptable; revisit if detectors are unified.
INFO	No circular dependencies found	`core` <- `data`/`app`; `app.desktop` <- `core`+`data`. Layering respected.	imports across modules	Keep.

Separation of concerns: strong. Engines calculate, render items draw, the service glues, the UI orchestrates. The context engine is deliberately decoupled from the day object it summarizes.

---

3. DATA PIPELINE AUDIT

3.1 Source & schema

- **Databento GLBX.MDP3 Trades**. Normalized columns: `ts, price, size, side, symbol` (`loader.py:35`).
- **Timestamp**: derived from `ts_recv` (`loader.py:167-168`). Verified tz-aware `datetime64[ns, UTC]`, monotonic non-decreasing, **12.9% of rows share a ts** with the previous row (legitimate multi-fills -- one aggressor sweeping several resting orders; correctly **not** deduplicated).
- *Finding [LOW]*: `ts_event` (matching-engine time) is the more correct clock for order flow than `ts_recv` (gateway receive time). Impact is sub-ms->low-ms, negligible at 1-min/5-min bars, but real for fine-grained tape speed and near-boundary bar assignment. Databento provides `ts_event`.
- **Aggressor** `side`: `A`=Ask=sell aggressor, `B`=Bid=buy aggressor, `N`=unknown (`delta_engine.py:9-16`). **Verified correct** against the data: A~50.1%, B~49.9%, **N~0.0%**. Because N is negligible, delta's exclusion of N-volume is immaterial (buy+sell ~ total).

3.2 Session boundary, timezone, contract

- **Session**: 22:00 UTC -> 22:00 UTC (`loader.py:30`, `SESSION_START_UTC_HOUR=22`). Comment states "18:00 ET = 22:00 UTC (EDT)".
- *Finding [LOW/LATENT]*: this is a **fixed** offset. 18:00 ET = 22:00 UTC only under **EDT (summer)**; under **EST (winter)** it is 23:00 UTC. The May-Aug dataset is all EDT, so **correct now**; a winter file would mis-bin the boundary hour. `session_profiles` for Asia/London/NY *does* use `zoneinfo` (DST-correct) -- only the top-level futures boundary is hard-coded.
- **Contract selection**: `max(volume_per_symbol)` per file (`loader.py:219-220`) or globally in `load_many` (`:255-256`). **Verified**: the dataset spans a **rollover** -- NQM6 (June) through ~2026-06-07, NQU6 (Sept) from ~2026-06-18. Per-day loads pick the correct active contract. Spread symbols (`NQM6-NQU6`, etc.) exist but are filtered out because the outright dominates.
- *Finding [MEDIUM] -- composite across rollover*: `load_many` picks **one** global-max symbol and filters to it. For the **90D composite** anchored at 2026-08-06, the 57-file block (2026-05-08...08-05) resolves to **NQU6**, and **12 NQM6 days (May-early June) are dropped** -> the "90D" profile is built from **~45 days**, not 90. The 15D composite (single contract) is unaffected. Values aren't corrupted (single-contract price basis), but the **span is mislabeled / incomplete**. *Verified with `check_composite_fast.py`.*

3.3 Bar aggregation & derived series

- **OHLC**: `price.resample(interval).ohlc()`, `volume=size.sum()`, then `dropna(subset=["open"])` drops empty bars (`data_service.py:509-513`).
- *Finding [LOW/LATENT]*: dropping empty bars makes `t` **index-contiguous but not guaranteed time-contiguous**; bar-count lookbacks would then span a real-time gap. **Verified**: on NQ at 1min and 5min there are **zero interior gaps** (every minute trades; the maintenance hour falls at the session edge), so no current impact. Would matter for a less-liquid instrument.

- **VWAP**: developing, volume-weighted typical price `(H+L+C)/3` (`:519-521`). Per-bar typical (not per-tick) -- standard approximation.
- **CVD**: `cumsum(per-bar (buy?sell))` from the footprint (`:562-563`). Causal cumulative. Uses B/A only (N dropped in `_build_footprint`), consistent with the delta engine.
- **Developing POC/VAH/VAL** (`developing_levels`, `:480-498`): accumulates each bar's cells into a VP engine and snapshots POC/VA **after each bar** -> `dev_poc[k]` sees **only** bars 0..k. This is the causal backbone and it is correct by construction. Final value == full-day value (tested).
- **tps** (trades/sec): per-bar print count / bar_seconds (`:578-584`). Causal per-bar.

3.4 Edge cases (as handled in code)

Case	Handling	Verdict
Zero volume bar	dropped by `dropna`; developing forward-fills last POC/VA	OK
Unknown aggressor `N`	excluded from delta/CVD; still in raw VP volume	OK (N~0 here)
Duplicate/multi-fill ts	kept (correct -- separate fills)	OK
Out-of-order trades	`sort_values("ts", stable)`	OK
Missing prev-day file (session)	`incomplete=True` flag set	OK
Empty file / no valid rows	raises `ValueError`	OK
Spread symbols	filtered by max-volume selection	OK (implicit)
Rollover	per-day correct; **composite drops minority contract**	**MEDIUM (see 3.2)**

---

4. ORDER-FLOW COMPONENTS (per-component audit)

For every component: **Purpose - Inputs - Algorithm (real code) - Output/States - Thresholds - Causality/Look-ahead - Edge cases - Determinism - Real-data validity - Interpretation**. Thresholds are **adaptive** unless noted.

P1a -- Structured VP Nodes (`vp_engine.compute_nodes`)

- **Purpose**: HVN/LVN as **zones** (low/high/width/prominence/tier), not single prices -- needed so price<->LVN interaction can ask "did it traverse the whole zone or reject at the edge?"
- **Algorithm**: local max/min within an adaptive window (`window_ratio=3%` of levels), prominence vs the **mean** profile volume (`min_prominence_ratio`), merge candidates within `merge_gap_ticks`, tier `major` if prominence >= `major_prominence_ratio=0.6`. `_node_candidates` is shared with `compute_hvn_lvn_peaks` (single source of truth).
- **States/Output**: `VolumeNode(kind, price, low, high, width, volume, prominence, tier)`.
- **Causality**: N/A (operates on whatever profile it's given; in the engine it's built from footprint <= T).
- **Edge**: prominence clamped >=0; LVN restricted to between HVNs by default (edges of the distribution are noise).
- **Note [LOW]**: prominence vs **mean** (skewed by the POC spike) rather than median; and POC tie-break (`max` over dict) is **insertion-order** dependent on exact-equal volume (rare; non-deterministic on ties).
- **Interpretation**: HVN = acceptance/value; LVN = thin zone price tends to reject or fast-traverse.

P4.1 -- Delta (`analyze_delta`, `context_engine.py:231`)

- **Purpose**: describe the **evolution** of per-bar delta (direction + magnitude + variation), never "positive = BUY".
- **Inputs**: `cvd` (causal); per-bar delta = `diff(cvd)`.
- **Algorithm**: `scale = median(|recent lookback=7|)`; `neutral = 0.5 - scale`; momentum from mean(recent); acceleration = |cur| vs |prev| with `accel_tol=0.15`; state via `_classify_delta_state` (order: NEUTRAL -> DELTA_FLIP -> ACCEL/DECEL -> AGGRESSION).
- **States (8)**: `NEUTRAL, DELTA_FLIP, {BUYING,SELLING}_{AGGRESSION,ACCELERATION,DECELERATION}`.
- **Thresholds**: adaptive (median of recent |delta|).
- **Causality/Look-ahead**: `diff(cvd)`, only <= T. [OK]
- **Edge**: n=0 -> NEUTRAL; scale<=0 -> falls back to |cur| (no div-by-zero).
- **Note [LOW]**: `DELTA_FLIP` triggers if `prev != 0` (not `|prev| >= neutral`), so a negligible opposite-sign prev can flag a flip.

- **Interpretation**: aggression strong/accelerating/decelerating/flipping -- effort of the aggressor, not a signal.

P4.2 -- Price Progress vs Delta (`analyze_price_progress`, `:338`)

- **Purpose**: **effort (delta) vs result (ticks)** on a short window. "Delta down + price down" is *not* auto-bearish; it measures how much effort produced how much progress.
- **Algorithm**: window `w=3`; needs `n >= 2w+1`; `agg_scale/progress_scale = median of past NON-overlapping windows` (adaptive); levels HIGH/MEDIUM/LOW via `high_ratio=1.3`/`low_ratio=0.6`.
- **States (6)**: `AGGRESSION_WITH/WITHOUT_PROGRESS, PROGRESS_WITHOUT_AGGRESSION, QUIET, NEUTRAL, INSUFFICIENT_EVIDENCE`. Also exposes `efficiency`, `direction_alignment` (ALIGNED/OPPOSED/FLAT -- exposed, **not** used in overall).
- **Causality**: past windows end at `j <= n-w-1`, no overlap with the current window; all <= T. [OK] (Verified index math.)
- **Interpretation**: `AGGRESSION_WITHOUT_PROGRESS` = aggressor is being stopped (a key tell); `PROGRESS_WITHOUT_AGGRESSION` = drift on little effort.

P4.3 -- Absorption / Exhaustion -- *see §5 for the deep-dive*

- **Absorption** (`analyze_absorption`, `:517`): most-recent closed-bar candidate with **dominance** (`dom=1.8`) + **concentration** (`frac=0.22`) + **extreme location** (bottom/top `zone=2` levels) + **rejection** (`reject=0.55`) + **adaptive min volume**; FORMING->CONFIRMED/FADED after `confirmation_window=3` (reaction read only when elapsed).
- **Exhaustion** (`analyze_exhaustion`, `:581`): **climax volume** (`vol_mult=2.0xmedian`) + **new extreme** over `window=14` + **delta in trend** (`delta_frac=0.20`), then reversal.
- **States**: 7 each (`NONE`, `{BULL/BEAR}_ABSORPTION_{FORMING/CONFIRMED/FADED}`; `{TOP/BOT}_EXHAUSTION_{...}`).
- **Causality**: reaction bars `k+1..k+w` read **only** when `bars_since >= w` (i.e. `k+w <= last`). [OK]

P4.4 -- CVD Divergence -- *see §6*

- `analyze_cvd_divergence` (`:704`): structural swings (`left=right=3`), pending swing -> FORMING; adaptive `price_ratio=0.5 - median(range)`, `cvd_ratio=0.3 - std(cvd)`; states `BULLISH/BEARISH/NO/INSUFFICIENT`. Causal (`_swings` uses `arr[k+1:k+R+1] <= last`). [OK]

P4.5 -- POC Migration -- *see §7*

- `analyze_poc_migration` (`:812`): direction/strength/speed/consistency of the **developing** POC over `window=10`; adaptive scale = mean per-bar POC displacement x window; states `RISING/FALLING/SIDEWAYS/INSUFFICIENT`. Causal (uses `poc_series` <= T). [OK]

P1b -- LVN Interaction & Acceptance/Rejection -- *see §8*

- Generic engine `analyze_level_interaction` (`:919`) on a zone `[zlo,zhi]`, reused for LVN nodes and for the nearest POC/VAH/VAL. States `TEST/REJECTION/ACCEPTANCE/FAST_TRAVERSAL/FAILED_REJECTION/INSUFFICIENT`. Causal + confirmation window. [OK]
- **Perf note [MEDIUM]**: `analyze_lvn_interaction` rebuilds a VP from the full <=T footprint on every call (§13).

P5 -- Tape Speed -- *see §9*

- `analyze_tape_speed` (`:1122`): `contracts_per_sec=vol/bar_seconds` classified vs adaptive scale (excludes current bar); `trades_per_sec`, `delta_per_sec` exposed **separately** (speed decoupled from direction). [OK]

P2 -- Previous Session Context -- *see §10*

- `analyze_session_context` (`:1214`): prev-session + intraday sub-session profiles gated by `available_from <= now`; price location vs value; nearest-level interaction. Causal. [OK]

P3 -- Composite Context -- *see §10*

- `analyze_composite_context` (`:1311`): 15D/90D structure + multi-timeframe agreement (transparent rules) + confluence. Causal (`available_from=0`, ends yesterday). [OK] *But see composite-rollover finding (§3.2).*

P6 -- Execution Context / Overall -- *see §11*

- `_execution_summary` (`:1421`): counts directional evidence **only** from delta/cvd/absorption/exhaustion; poc/tape/lvn/session/composite -> reasons only. Overall ? {SUPPORTIVE, NEUTRAL, CONTRADICTING, INSUFFICIENT}. Transparent, no scoring. [OK] (UX/semantics caveats in §11.)

5. ABSORPTION / EXHAUSTION AUDIT

Is `AGGRESSION_WITHOUT_PROGRESS` different from `ABSORPTION`? -- YES, by design and by test.

- `analyze_price_progress` measures effort-vs-result over a window with **no location or rejection requirement**.
- `_absorption_candidate` (`:496`) requires **all five**: dominance (`sell_lo >= 1.8 - buy_lo`), concentration (`sell_lo >= 0.22 - v_total`), extreme location (bottom/top 2 levels), rejection (`(close?low)/range >= 0.55`), adaptive min volume.
- **Test proof**: `test_aggression_without_progress_is_NOT_absorption` -- massive aggressive selling at the low but **close near the low** (no rejection) ? `detected=False`. [OK]

Does exhaustion stay distinct from absorption? -- YES.

- Absorption = rejection **within** the bar (wick). Exhaustion = **climax at a new extreme** then reversal over following bars.
- **Test proof**: `test_exhaustion_and_absorption_are_separate_components` -- an exhaustion scenario returns `absorption.detected=False`. [OK]

Structural false positives / caveats:

- **[LIMITATION -- INHERENT]**: with **trades-only** data (no order book / MBO), absorption is **inferred** from *aggressor-dominance + rejection*, never directly observed (you cannot see the passive limit orders doing the absorbing). This is the best achievable with the data, but it should be stated: it is an inference, not a measurement.
- **[MEDIUM -- overlap at extremes]**: bull-absorption and bot-exhaustion (and bear/top) describe **related "rejection at an extreme" phenomena** and can both fire around the same swing. They are separate mechanisms and often separate bars, but they are **correlated evidence** -- which matters for the `overall` count (§11, finding #4).
- **[NEEDS VALIDATION]**: thresholds (`frac=0.22` concentration over 2 levels, `dom=1.8`) were tuned for the NY open; the handoff notes ~30 absorption events/day -- possibly too permissive for "notable event". Not tunable on isolated examples; validate on the full set.

6. CVD DIVERGENCE AUDIT

- **Swing detection**: `_swings` (`:663`) -- confirmed local extremes with `L=3` left and `R=3` right bars. **Structural**, not consecutive-bar slope.
- **Left/right confirmation & pending**: a confirmed swing needs `R` bars to its right; the most recent unconfirmed extreme is a **pending** swing -> status **FORMING**; becomes **CONFIRMED** once `R` bars elapse.
- **CVD reference & adaptive thresholds**: price gap vs `0.5 - median(range)`, CVD gap vs `0.3 - std(cvd)`; strength = `|Deltacvd|/std(cvd)` (a magnitude, **not** a trade score).
- **Timeframe dependency**: divergence is computed on the same bars as everything else (the chosen interval).
- **Look-ahead**: `_swings` uses `arr[k+1:k+R+1] <= last`; `_pending_swing` uses `arr[k+1:last+1]`. [OK]
- **Verdict -- is it a real divergence or a math approximation?** It is a **genuine structural divergence** (HH price + LOWER cvd = bearish; LL price + HIGHER cvd = bullish), with adaptive noise floors. **Test proofs**: `test_structural_not_consecutive_bars` (per-bar zigzag ? not confirmed), `test_weak_noisy_divergence_is_not_flagged` (sub-threshold ? NO_DIVERGENCE), `test_adaptive_scaling_same_result_x10` (scale-invariant), real-data smoke shows divergences appear.
- **Limitation**: it compares only the **last two** confirmed swings of each kind -- multi-leg / longer-range divergences are not captured, and any two adjacent qualifying swings can trigger it.

7. POC MIGRATION AUDIT

- **Developing, not full-day**: uses `snapshot.poc_series` (`:816`) = the developing POC trail. **Test proof**: `test_poc_migration_uses_developing_not_full_day` asserts `poc_current==118` (last developing), not the full-day 999. [OK] (And `snapshot_from_daydata` never passes `day.poc` -- it slices `dev_poc[k]`.)

- **poc_series / session accumulation**: cumulative from session open; `net = poc[-1]?poc[-w-1]`, `consistency = |net|/total_path`, `speed = |net|/w/tick`.
- **Sideways / adaptive threshold**: SIDEWAYS if `bar_disp<=0` or `|net| < 0.35 - (mean per-bar displacement x window)`; STRONG if `consistency >= 0.6`.
- **Look-ahead**: slices `poc_series[-(w+1):] <= T`. [OK]
- **[LIMITATION -- sensitivity, UNKNOWN]**: the developing POC becomes **"sticky"** as session volume accumulates -- late in the session a single bar's volume is tiny vs the accumulated profile, so the POC barely moves and the component reads **SIDEWAYS** most of the time. The adaptive scale partially compensates (it shrinks too), but this component is likely **near-uninformative in many late-session windows**. This sensitivity concern is **not tested** (synthetic tests feed hand-crafted `poc_series`) and remains an open empirical question. This is the weakest of the 11 components on expected value.

---

8. LVN / ACCEPTANCE / REJECTION AUDIT

- **Zone, not price -- YES.** `analyze_lvn_interaction` builds `VolumeNode`s (low/high/tier) and drives `analyze_level_interaction(zlo, zhi, ...)`. Point levels (POC/VAH/VAL) get a level-specific adaptive band (`point_band_ratio` -- POC wider as a magnet).
- **Mechanics** (`:919-1021`): finds runs of bars touching the zone (`tol_ticks=1`), takes the last run; computes approach (FROM_ABOVE/BELOW from `close[rs-1]`), penetration, bars_inside, exit (BACK/THROUGH/INSIDE), move_away, delta_during, prior_rejection.
- **Rules (microstructurally sensible):**
- still inside & `bars_inside >= 1.5 - expected_cross` -> **ACCEPTANCE**; else **TEST/FORMING**.
- exit BACK & `move_away >= 0.6 - bar_range` -> **REJECTION** (CONFIRMED once `post >= R`).
- exit THROUGH & `bars_inside <= 0.6 - expected_cross` -> **FAST_TRAVERSAL**; else **ACCEPTANCE**.
- prior rejection later accepted -> **FAILED_REJECTION**.
- `expected_cross = zone_width_ticks / bar_range_ticks` -- a genuine zone-width-aware threshold.
- **Look-ahead**: uses bars <= last; `status` CONFIRMED only after `post >= R`. [OK]
- **Coverage**: all 5 states tested (`test_level_interaction.py`).
- **Verdict**: the rules make microstructural sense and treat the LVN as a real zone.

---

9. TAPE SPEED AUDIT

- **Separate metrics**: `trades_per_sec` (churn, from `tps`), `contracts_per_sec` (throughput = vol/bar_seconds, the primary speed metric), `delta_per_sec` (direction).
- **Speed NOT contaminated by direction -- YES.** `speed_level` is computed only from `contracts_per_sec` vs an adaptive scale; `delta_per_sec` is exposed but never enters the level.
- **Test proof**: `test_speed_independent_of_delta_direction` -- identical volume with **opposite** delta signs ? **same** `speed_level=HIGH`, opposite `delta_per_sec`. [OK] So *HIGH speed + weak delta* and *HIGH speed + strong delta* are correctly represented as (same speed, different delta) rather than merged into one verdict.
- **Adaptive scale** excludes the current bar (`[...:last]`) -- avoids self-normalization.
- **Data honesty**: comment correctly notes there is **no orders/sec** (no MBO/order book), only prints/sec and contracts/sec -- both reliable and different.
- **[NEEDS VALIDATION]**: `high_ratio=1.4` -> handoff notes ~13% of bars flagged HIGH; possibly permissive for "notable".

---

10. PREVIOUS SESSION / COMPOSITE -- NO-LOOK-AHEAD PROOF

Previous session (P2):

- `build_reference_levels` (`data_service.py:601`): prev **completed** session -> `available_from=0` (fully in the past, always known). Today's sub-sessions (Asia/London/NY) -> `available_from = session close epoch`.

- `analyze_session_context` (`:1222`) filters `available_from <= now`. So **Asia becomes context only after Asia closes**, London after London closes, etc. A completed session's POC/VA *is* known at its close -> revealing it then is causal, not look-ahead. [OK]
- Sub-session windows use `zoneinfo` (DST-correct).

Composite (P3):

- `build_composite_levels` (`:661`): anchored at `prev_date` (yesterday) and spans **N days ending yesterday** -- **today is excluded**. `available_from=0`. [OK] Current-day exclusion confirmed in code and by construction.
- `analyze_composite_context` filters `available_from <= now`.

"Does information become available exactly when it should?" -- YES, with the single caveat that the **90D composite silently under-covers across the rollover** (§3.2) -- a *data-coverage* issue, not a look-ahead issue.

---

11. EXECUTION CONTEXT / OVERALL AUDIT

Rule (`_execution_summary`, `:1421-1481`), audited line by line:

- Directional evidence counted **only** from: delta (`BUYING/SELLING_AGGRESSION|ACCELERATION`), cvd_divergence (CONFIRMED only), absorption (CONFIRMED, kind->bull/bear), exhaustion (CONFIRMED, bot->bull / top->bear). DECELERATION and FORMING do **not** count.
- poc/tape/lvn/session/composite -> **reasons text only**, never leans.
- Overall: `INSUFFICIENT` if pp & cvd both insufficient; `CONTRADICTING` if bull>0 and bear>0; `SUPPORTIVE` if (bull>=2, bear=0) **or** (bear>=2, bull=0); else `NEUTRAL`.

Assessment against the requirements:

Requirement	Verdict
Deterministic	[OK] pure function; test `test_summary_deterministic`.
Transparent (no hidden scoring)	[OK] every reason is emitted, last line is the literal rule "evidence: X bullish / Y bearish -> overall".
No implicit BUY/SELL	[OK] headline is SUPPORTIVE/NEUTRAL/CONTRADICTING/INSUFFICIENT -- no direction.
No single-component over-weight	[OK] each contributes at most 1 (absorption/exhaustion add a boolean).
Doesn't confuse context with signal	[OK] non-directional components are reasons only.

Findings:

- **[MEDIUM -- misleading color]** The UI colors `SUPPORTIVE` **green** (`theme.UP`) and `CONTRADICTING` **red** (`main.py:2073`). But `SUPPORTIVE` fires for **both** 2-bullish and 2-bearish coherence. A strongly *bearish* context therefore renders as a **green "SUPPORTIVE"** headline. Green=long is the universal trading convention -> real misread risk. The direction is only in the smaller `reasons` list. *Recommendation*: decouple the color from "coherence" (e.g. neutral/blue for SUPPORTIVE), or rename to `COHERENT`/`CONFLICTING`, or surface the actual lean.
- **[MEDIUM -- correlated evidence]** bull-absorption + bot-exhaustion at a low both increment `bull` -> `bull>=2` -> SUPPORTIVE from essentially one "rejection at a low" phenomenon. **Codified as intended** by `test_supportive_from_absorption_and_exhaustion` -- a textbook "tests pass != concept correct" case. *Recommendation*: treat absorption+exhaustion at the same extreme/side as <=1 combined vote.
- **[OK]** Requiring >=2 one-sided pieces for SUPPORTIVE (single piece -> NEUTRAL) is a reasonable coherence bar.

Is `overall` justified or misleading? The *logic* is sound and transparent; the **presentation** (green SUPPORTIVE) and the **evidence-independence assumption** are the two things that can mislead. On its own, `overall` adds little new information beyond the components -- it is a coherence indicator, not a source.

---

12. UI AUDIT

- **Clean Chart**: markers (Big Trades/Absorption/Exhaustion) start **OFF** (`test_execution_context_panel_renders` asserts this). Engine computes everything; overlays are opt-in. Good.
- **Execution Context panel** (`_render_execution_html`, `:2067`): sections FLOW/STRUCTURE/LEVELS/LOCATION, human-readable sentences, **hides NONE**, Technical Details toggle for raw fields, disclaimer "Order flow context

only. Your price-action setup decides execution." Clean and on-philosophy.

- **Context-at-Cursor**: throttled -- recomputes only when the cursor crosses into a **different** bar (`:1294`), causal (`upto_index=i`).
- **Findings:**
- [MEDIUM] Green "SUPPORTIVE" color semantics (see §11).
- [LOW/latent] Cursor->bar index assumes uniform spacing (`:1292`); verified no interior gaps on NQ, so no current impact.
- [INFO] Views (`Curat/Order Flow/NY Open/Tot`) are sensible presets; "Tot" is intentionally dense (opt-in). The default (Curat) is clean.
- **Verdict**: the UI communicates context efficiently to a discretionary trader and resists dashboard-clutter by defaulting to clean + hiding NONE. The one real risk is the color of the overall headline.

---

13. PERFORMANCE AUDIT

Area	Complexity	Note
Footprint render	O(visible cells)	**Well optimized** -- viewport culling, pre-allocated brush pools, pixel-size gating for numbers/bars (`charts.py:309-437`).
`developing_levels`	O(bars x avg levels)	Computed **once** per day load; VP result recomputed each bar (acceptable one-off).
`analyze_lvn_interaction`	O(bars x levels) **per call**	**[MEDIUM]** Rebuilds a full VP from the <=T footprint on **every** context evaluation (`context_engine.py:1024-1035`). On a full 1-min session on hover this is the most likely lag source. Throttled to once-per-bar-crossing, but still a full VP rebuild each time. *Recommendation*: cache LVN nodes per (day, resolution) and reuse; or compute incrementally in replay. **Measure on the real GPU/session first.**
Composite 90D	O(90 files load)	Slow file I/O; correctly **kept out of the live panel** (only 15D is live; `_ensure_ctx_levels` uses `spans=(15,)`).
Replay seek	O(ticks) with checkpoints every 40 candles	Reasonable; deepcopy at checkpoints.
Context on hover	full 11-component analyze per bar-cross	Dominated by the LVN VP rebuild above.

No memory leaks observed; render items reuse pictures. The main measurable risk is the LVN VP rebuild.

---

14. TEST AUDIT ("tests pass" != "logic correct")

~249 test functions across 28 files -> **261 test cases, all passing** (`pytest` exit 0 at `b3217da`; runtime **2h20m** -- long because several tests hit real data and the 90D composite, itself a mild smell for CI). **Quality is genuinely high** -- many tests validate the *concept*:

- **Causality**: the `test_engine_result_identical_whether_or_not_future_exists` pattern (lookahead, absorption, exhaustion, cvd, execution, tape, poc) is the *correct* conceptual test -- the result at T is identical whether the day has future bars or ends at T. The synthetic fixture sets full-day POC=999 vs developing=200+k so any accidental full-day use fails loudly.
- **Counter-examples**: `test_aggression_without_progress_is_NOT_absorption`, `test_exhaustion_and_absorption_are_separate_components`, `test_structural_not_consecutive_bars`, `test_speed_independent_of_delta_direction`.
- **Robustness**: adaptive scale-invariance (`x10`), noise rejection, determinism, "state always in known set".

Gaps / weak tests (flagged):

Gap	Detail
Developing intermediate values not independently verified	`test_developing` checks endpoint + ordering + self-consistency, but **not** `dev_poc[k] == VP(footprint 0..k).poc` for interior k. (Causality holds by inspection; a direct interior test would harden it.)
Threshold sensitivity untested	Inherently empirical; no test can validate that 0.22 concentration or 1.4 tape ratio is "right".

Gap	Detail
Overall double-count codified as intended	`test_supportive_from_absorption_and_exhaustion` **asserts** the correlated-evidence behavior -- validates the implementation, not the concept (see §11 #4).
UI color semantics untested	Nothing checks that green SUPPORTIVE can be bearish.
No statistical validation	The harness (below) is exploratory, not a validation of edge.

Validation harness (`scripts/validate_context.py`) -- honest but not statistical:

- Causal (`context_at` slices at i; `outcome_after` looks ahead **only** for labeling). [OK]
- **But**: no baseline/unconditional base-rate; single 5-bar horizon; 6 days; 12 categories with no multiple-comparison correction; close-to-close only (no MFE/MAE). The ~48-52% follow-through **looks like chance** but, without a base rate, even "it's noise" is not established. Dead-code `total_bars` placeholder (`:170`).

15. REDUNDANCY / OVERENGINEERING

Pairwise assessment of the 11 components:

Pair	Relationship	Verdict
delta <-> price_progress	delta = raw aggression; price_progress = aggression vs result	**Distinct** (result axis adds info)
price_progress <-> absorption	absorption adds location+rejection+concentration	**Distinct** (tested)
absorption <-> exhaustion	both "rejection/reversal at an extreme"	**Overlap** -- correlated at extremes; can double-count in overall
cvd_divergence <-> delta	divergence is structural (price vs cvd swings); delta is per-bar	**Distinct**
poc_migration <-> (others)	developing value drift	**Distinct** but low-value (inertia)
lvn_interaction <-> acceptance_rejection	**same generic engine**, different zones (LVN node vs POC/VAH/VAL)	**Complementary, not redundant** (shared code by design)
session_context <-> composite_context	same "location vs value area" idea, different timeframes	**Complementary**; but composite's `SUPPORTIVE/CONTRADICTING` label overlaps the overall vocabulary (confusing)
tape_speed <-> (all)	activity axis, decoupled from direction	**Distinct**

Code redundancy (real): `detect_absorption/detect_exhaustion` (data_service, fixed thresholds) duplicate `analyze_absorption/analyze_exhaustion` (engine, adaptive). Two implementations of the same concept -> maintenance + disagreement risk.

Conclusion: you have **~9-10 genuinely distinct information sources**, not 11. Nothing is egregiously over-engineered; the main artificial-contradiction risk is absorption+exhaustion feeding `overall` as if independent.

16. REAL TRADING USEFULNESS (discretionary; NOT statistically validated)

> *No component is asserted "profitable". Classification is a *prior* on likely usefulness for a discretionary reader, pending validation.*

Tier	Components	Rationale
Likely HIGH	price_progress (effort vs result), absorption/exhaustion **at levels**, tape_speed, session/composite **location vs value**	Direct microstructural reads hard to eyeball; well-separated concepts
Likely MEDIUM	delta evolution, lvn_interaction, acceptance_rejection, cvd_divergence	Useful but noisier / thresholds sensitive
Likely LOW	poc_migration (developing-POC inertia), overall (a coherence summary, not a new source)	Low sensitivity / derivative
UNKNOWN -- REQUIRES VALIDATION	**all of the above, as predictive value**	No base-rate-relative evidence exists yet

17. BUG REPORT

Format: ID - Severity - File - Symptom - Why wrong - Repro - Expected - Actual - Recommendation.

***No bug produces incorrect numbers on the current dataset.** The two "latent" items are the closest to real bugs.*

BUG-1 - LOW/LATENT - `data/loader.py:30` (SESSION_START_UTC_HOUR=22)

Fixed 22:00 UTC session boundary assumes EDT. In EST (winter) the CME 18:00 ET boundary is 23:00 UTC. *Repro*: load a Nov-Feb file -> the 22:00-23:00 UTC hour is assigned to the wrong session. *Expected*: DST-aware boundary (via `zoneinfo` on `America/Chicago`/`New_York`). *Actual*: off-by-one-hour boundary in winter. *Impact now*: none (May-Aug all EDT). *Fix*: derive the boundary with `zoneinfo`, like `session_profiles` already does.

BUG-2 - LOW/LATENT - `app/desktop/main.py:1292`

`i = round((snapped ? t[0]) / bar_seconds)` assumes contiguous bars; `load_day` drops empty bars, so an interior gap would map the cursor to the wrong bar's context. *Repro*: a session with an interior empty bar. *Expected*: index by nearest `t` (searchsorted). *Actual*: index offset after a gap. *Impact now*: **none** -- verified 0 interior gaps on NQ 1min/5min. *Fix*: `np.searchsorted(d.t, snapped)` / nearest-epoch lookup.

BUG-3 - LOW - `data_service.py` vs `context_engine.py` (dual absorption/exhaustion)

Chart markers use fixed thresholds (`ABS_MIN_VOL=70`...) while the panel uses adaptive thresholds. *Symptom*: a marker can show absorption where the panel says NONE (or vice-versa). *Fix*: unify (markers consume engine output) or label them as distinct views.

BUG-4 - VERY LOW - `vp_engine.py:113` (POC tie-break)

`max(self.profile.items(), key=vol)` returns the first max in **dict insertion order** on exact-equal volume -> POC can depend on tick arrival order (mostly relevant to tiny synthetic/early-developing profiles). *Fix*: deterministic tie-break (e.g. nearest to prior POC, or lowest price).

BUG-5 - COSMETIC - `scripts/validate_context.py:170`

`total_bars = sum(1 for _ in [0])` dead placeholder. *Fix*: remove.

(No off-by-one, sign, timezone-parse, aggressor-mapping, stale-cache, or NaN-propagation bugs were found in the causal/calculation paths.)

---

18. FINAL SCORECARD

Correctness = calculation correct - Causality = no look-ahead - Tests = coverage+quality - Real-data = validated on real data - Complexity - Usefulness (discretionary prior) - Risk.

Component	Correctness	Causality	Tests	Real-data	Complexity	Usefulness	Risk	Recommendation
VP nodes (P1a)	GOOD	N/A	GOOD	PARTIAL	MED	MED	LOW	KEEP
Delta (P4.1)	GOOD	GOOD	GOOD	PARTIAL	LOW	MED	LOW	KEEP + VALIDATE
Price Progress (P4.2)	GOOD	GOOD	GOOD	PARTIAL	MED	**HIGH**	LOW	KEEP + VALIDATE
Absorption (P4.3)	GOOD	GOOD	GOOD	PARTIAL	MED	HIGH	MED	KEEP + VALIDATE + de-correlate from exhaustion
Exhaustion (P4.3)	GOOD	GOOD	GOOD	PARTIAL	MED	HIGH	MED	KEEP + VALIDATE + de-correlate
CVD Divergence (P4.4)	GOOD	GOOD	GOOD	PARTIAL	MED	MED	LOW	KEEP + VALIDATE
POC Migration (P4.5)	GOOD	GOOD	GOOD	PARTIAL	MED	**LOW**	LOW	KEEP but LOW priority; validate sensitivity
LVN Interaction (P1b)	GOOD	GOOD	GOOD	PARTIAL	HIGH	MED	MED (perf)	KEEP + cache VP
Acceptance/Rejection (P1b)	GOOD	GOOD	GOOD	PARTIAL	MED	MED	LOW	KEEP
Tape Speed (P5)	GOOD	GOOD	GOOD	PARTIAL	LOW	HIGH	LOW	KEEP + VALIDATE thresholds
Session Context (P2)	GOOD	GOOD	GOOD	PARTIAL	MED	HIGH	LOW	KEEP

Component	Correctness	Causality	Tests	Real-data	Complexity	Usefulness	Risk	Recommendation
Composite Context (P3)	**PARTIAL**	GOOD	GOOD	PARTIAL	MED	MED	MED	FIX rollover coverage
Overall (P6)	GOOD (logic)	GOOD	GOOD	PARTIAL	LOW	LOW	MED (UX)	FIX color + de-correlate evidence
Data pipeline	GOOD	GOOD	GOOD	GOOD	MED	--	LOW (BUG-1 latent)	KEEP; DST-proof boundary
Replay	GOOD	GOOD	GOOD	GOOD	HIGH	--	LOW	KEEP
Validation harness	GOOD (causal)	GOOD	PARTIAL	GOOD	LOW	--	--	UPGRADE to statistical

19. PRIORITY LISTS

MUST FIX (blocking correctness/interpretation for real use)

1. **Composite rollover coverage** (`build_composite_levels`) -- either back-adjust across contracts or **label the effective day-count** so "90D" isn't misleading. (MEDIUM)
2. `overall` **green "SUPPORTIVE" semantics** -- decouple color from coherence or surface the actual lean; a green headline on bearish coherence is genuinely misleading. (MEDIUM)

SHOULD IMPROVE

3. **Unify the two absorption/exhaustion implementations** (engine vs legacy markers). (MEDIUM)
4. **De-correlate absorption+exhaustion in** `overall` (<=1 combined vote per extreme/side). (MEDIUM)
5. **Upgrade the validation harness** to base-rate-relative, multi-horizon, all-62-days, MFE/MAE. (enables everything else)
6. **DST-proof the session boundary** (BUG-1). (LOW, before any winter data)
7. **Cache LVN nodes** to remove the per-hover VP rebuild (BUG/perf). (LOW-MED)

OPTIONAL

8. `ts_event` instead of `ts_recv`. 9. Deterministic POC tie-break. 10. Cursor->index via searchsorted. 11. Remove dead code / legacy scripts / streamlit. 12. Rename composite `SUPPORTIVE/CONTRADICTING` to avoid clashing with the overall vocabulary. 13. "bars since" in the panel for persistent events.

DO NOT TOUCH (stable; don't complicate)

- The **Snapshot / causality boundary** and `snapshot_from_daydata` -- it's the crown jewel; changing it risks the one thing that's provably right.
- **Replay** (tick-by-tick + checkpoints) -- correct and tested.
- **Delta / VP / Value-Area engines** -- clean and correct.
- The **individual component algorithms** -- do not tune thresholds on isolated examples; validate first.

20. RECOMMENDED NEXT STEPS (ordered, with rationale)

1. **Statistical empirical validation FIRST** (not code). *Why*: you cannot correctly decide what to keep, tune, or cut until you know each component-state's lift over the unconditional base rate. Tuning or refactoring before this risks optimizing noise. Deliverable: per-component, per-state follow-through across >=3 horizons vs base rate, all 62 days, with MFE/MAE and event counts.
2. **Then the two MUST-FIX items** (composite label, overall color) -- cheap, remove active misinterpretation.
3. **Then de-correlate/unify** (absorption+exhaustion evidence; the dual detectors) -- informed by (1).
4. **Then perf** (LVN cache) -- only if (1) keeps LVN interaction.
5. **UI refinement / replay study** -- continuous, low-risk.
6. **DST fix** -- before ingesting any winter data.

Do not start with refactoring or performance; start with **measurement**.

21. COMPLETE COMPONENT REFERENCE

Read this in 6 months to understand each tool without the code. Thresholds are defaults (all overridable via `ContextEngine(config=...)`).

Snapshot (the causal input)

- **Purpose**: carry *only* <=T information. **Process**: `snapshot_from_daydata(day, upto_index=k)` slices all arrays to [0..k], filters footprint to epochs <= T, takes `dev_poc[k]/dev_vah[k]/dev_val[k]` and `poc_series`/`tps` (developing). **Output**: frozen `Snapshot`. **Causality**: physical truncation -- the engine cannot see the future because it isn't in the object. **Look-ahead risk**: none (proven by "identical with/without future" tests). **Status**: FROZEN -- do not touch.

Delta (P4.1)

Input: cvd. Process: per-bar `diff(cvd)`, adaptive `scale=median(|recent7|)`, direction+momentum+acceleration. Output: 8 states + scale/sequence. Thresholds: `neutral 0.5 - scale`, `aggression 1.2 - scale`, `accel_tol 0.15`. Interpretation: strength/behaviour of aggression. Look-ahead: none. Tests: `test_delta_component.py` (20).

Price Progress (P4.2)

Input: close, cvd. Process: effort `|Sumdelta|` vs result `|Deltaprice|` on w=3, adaptive scale from past non-overlapping windows. Output: 6 states + efficiency + alignment. Thresholds: `high 1.3 - scale`, `low 0.6 - scale`. Interpretation: is aggression delivering movement? Look-ahead: none. Tests: `test_price_progress.py` (14).

Absorption (P4.3)

Input: footprint, OHLC, vol. Process: most-recent closed-bar with dominance+concentration+extreme-location+rejection+adaptive-min-vol; confirm after 3 bars. Output: 7 states + reaction ticks. Thresholds: `dom 1.8`, `frac 0.22`, `reject 0.55`, `min_vol_ratio 0.25`, `confirm 3`. Interpretation: aggressive side absorbed at an extreme -> potential support/resistance. **Limitation**: inferred (no order book). Look-ahead: none. Tests: `test_absorption*.py`, `test_absorption_exhaustion.py`.

Exhaustion (P4.3)

Input: OHLC, vol, cvd. Process: climax vol (2x median) + new extreme (window 14) + delta-in-trend (0.20), then reversal; confirm after 3. Output: 7 states. Interpretation: last impulse into an extreme -> potential reversal. **Overlaps absorption at extremes.** Look-ahead: none.

CVD Divergence (P4.4)

Input: high, low, cvd. Process: structural swings (L=R=3), pending->FORMING; adaptive price/cvd thresholds. Output: BULLISH/BEARISH/NO/INSUFFICIENT + strength + refs. Interpretation: price makes a new extreme, CVD doesn't. **Limitation**: last-two-swings only. Look-ahead: none. Tests: `test_cvd_divergence.py` (15).

POC Migration (P4.5)

Input: poc_series (developing). Process: net/consistency/speed over window 10, adaptive scale. Output: RISING/FALLING/SIDEWAYS + STRONG/WEAK. Interpretation: is value migrating? **Limitation**: developing-POC inertia -> low sensitivity late-session (UNKNOWN value). Look-ahead: none.

LVN Interaction / Acceptance-Rejection (P1b)

Input: footprint (LVN nodes) / POC-VAH-VAL; OHLC, cvd. Process: generic zone-interaction engine (touch runs, approach, penetration, exit, reaction) with confirmation window. Output: TEST/REJECTION/ACCEPTANCE/FAST_TRAVERSAL/FAILED_REJECTION. Interpretation: how price behaves at a thin zone / key level. **Perf**: LVN rebuilds VP per call. Look-ahead: none. Tests: `test_level_interaction.py` (18), `test_composite_lvn.py`.

Tape Speed (P5)

Input: vol, cvd, tps, bar_seconds. Process: contracts/sec vs adaptive scale (HIGH/NORMAL/LOW) + acceleration; trades/sec & delta/sec exposed separately. Output: speed level + acceleration. Interpretation: activity/urgency, **decoupled from direction**. Look-ahead: none. Tests: `test_tape_speed.py` (13).

Session Context (P2)

Input: reference_levels (prev + Asia/London/NY), close. Process: `available_from <= now` gate; location vs each session's value; nearest-level interaction. Output: profiles + location + nearest interaction. Interpretation: where price sits vs prior sessions. Look-ahead: none (gated). Tests: `test_session_context.py` (13), `test_prior_levels.py`.

Composite Context (P3)

Input: composite_levels (15D/90D), reference_levels. Process: multi-timeframe location agreement (transparent rules) + confluence near price. Output: SUPPORTIVE/NEUTRAL/CONTRADICTING (agreement) + reasons. Interpretation: long-term structure agreement. **Bug**: 90D under-covers across rollover. Look-ahead: none (ends yesterday). Tests: `test_composite_context.py` (17).

Overall (P6)

Input: the 11 components. Process: count one-sided CONFIRMED directional evidence (delta/cvd/absorption/exhaustion only); >=2 one-sided->SUPPORTIVE, both sides->CONTRADICTING, else NEUTRAL/INSUFFICIENT. Output: 4 states + reasons. Interpretation: **coherence** of evidence, not direction. **Caveats**: green-color semantics; correlated evidence. Look-ahead: none. Tests: `test_execution_context.py` (15).

End of audit. All findings are grounded in the code at `b3217da` and in read-only inspections of the parquet dataset; nothing in the repository was modified.